

# TOICA: 작업의 실행 위치와 채널 수를 함께 정하기

Joint Task Offloading and Channel Allocation in Spatial-Temporal Dynamic  
for MEC Networks

---

Tianyi Shi, Tiankui Zhang, Jonathan Loo, Rong Huang, Yapeng Wang (2025)

이동하는 단말의 의존 작업을 강화학습과 동적 계획법으로 연결한다

# 서버가 빠르다고 오프로딩도 빠른 것은 아니다

작업 순서·무선 전송·서버 내부 간섭이 함께 완료 시간을 바꾼다

| 요인       | 무엇이 달라지나     | 필요한 판단      |
|----------|--------------|-------------|
| 작업 의존성   | 앞 작업의 결과가 필요 | 실행 가능한 순서   |
| 단말 이동    | 거리·채널 이득 변화  | 어느 서버에 보낼지  |
| 공유 통신    | 같은 주파수의 간섭   | 채널을 얼마나 줄지  |
| 서버 병렬 실행 | VM 사이 I/O 간섭 | 한 서버에 몰릴 비용 |

오프로딩 위치만 고르고 통신 자원을 무작위로 주면 좋은 선택도 비싸질 수 있다

# 먼저 구분할 네 가지 이름

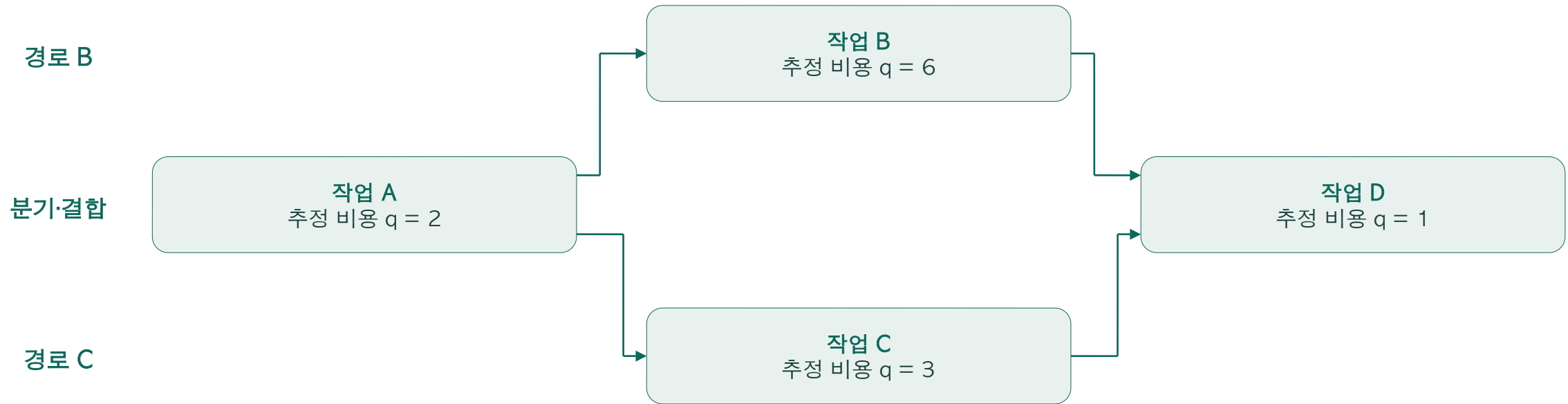
서버 선택과 채널 배분은 같은 행동 변수가 아니다

| 표기      | 뜻                           | 이 논문의 역할       |
|---------|-----------------------------|----------------|
| MT / ES | 이동 단말 / 엣지 서버               | 결정 주체 / 실행 자원  |
| DTO     | D3QN-based Task Offloading  | 로컬 또는 서버 선택    |
| DCA     | DP-based Channel Allocation | 서버별 채널 수 배분    |
| TOICA   | DTO와 DCA의 결합                | 배분 결과까지 보상에 반영 |

DTO가 서버를 고르면, 그 서버에 모인 단말들을 대상으로 DCA가 작동한다

# 의존 관계: 화살표는 선행 작업 완료를 요구한다

설명용 DAG: A 다음 B·C, 둘 다 끝난 다음 D가 실행 가능하다



D의 준비 시점은 B와 C의 완료 시점 중 더 늦은 시점이다

# 우선순위: 뒤에 남은 큰 비용을 역방향으로 누적한다

종료 작업은  $P=q$ , 나머지는  $P=q+\max(\text{후속 작업의 } P)$

| 작업    | 계산                                            | 우선순위 P        |
|-------|-----------------------------------------------|---------------|
| D     | 종료: $qD$                                      | 1             |
| B / C | $6+1$ / $3+1$                                 | $7$ / $4$     |
| A     | $2+\max(7,4)$                                 | 9             |
| 내림차순  | $A \rightarrow B \rightarrow C \rightarrow D$ | 의존성을 지키는 한 순서 |

우선순위는 작업 순서를 정하는 규칙이며, 실행 위치를 고르는 DTO와 역할이 다르다

# 슬롯 길이: 단말들의 처리시간 중 최댓값

논문은 단말별 현재 작업 하나를 한 슬롯 안에 끝내는 모델을 사용한다

| 슬롯 | 두 단말의 처리시간    | 슬롯 길이 $D(t)$   |
|----|---------------|----------------|
| 1  | 0.05s / 0.08s | 0.08s          |
| 2  | 0.02s / 0.04s | 0.04s          |
| 합계 | 작업시간 합은 0.19s | 슬롯 길이 합은 0.12s |

각 단말의 시간을 줄이는 것과 시스템의 가장 긴 시간을 줄이는 것은 다를 수 있다

# 두 종류의 간섭은 서로 다른 단계에 작용한다

같은 셀에서는 채널을 나누고, 이웃 셀은 같은 채널을 재사용할 수 있다

| 간섭        | 발생 조건           | 처리시간에 미치는 영향  |
|-----------|-----------------|---------------|
| 셀 간 무선 간섭 | 다른 셀의 동일 채널 사용  | SINR·전송률 감소   |
| 같은 셀의 채널  | 한 채널에 최대 한 단말   | 셀 안의 중복 배정 금지 |
| 서버 I/O 간섭 | 여러 단말의 VM 동시 실행 | 계산시간 증가       |
| 이동·페이딩    | 거리와 무선 상태 변화    | 같은 배분도 성능 변화  |

DCA는 채널 수를 배분하지만, 그 사실만으로 셀 간 간섭을 제거하지는 않는다

# 계산 예: 서버 혼잡은 무선이 같아도 비용을 바꾼다

설명용: 입력 2Mbit, 20Mbit/s, 50Mcycles, 서버 5GHz; 송신 1W·대기 0.3W

| 조건      | 전체 처리시간                 | 단말 에너지                    |
|---------|-------------------------|---------------------------|
| VM 1개   | $0.10+0.010 = 0.110s$   | $0.10+0.003 = 0.103J$     |
| VM 3개   | $0.10+0.0144 = 0.1144s$ | $0.10+0.00432 = 0.10432J$ |
| 계산시간 배율 | $(1+0.2)^2 = 1.44$      | 서버 전력은 제외                 |

엣지 실행 에너지는 단말의 송신·대기 에너지이며 시스템 전체 에너지가 아니다

# 목표: 지연과 단말 에너지의 가중 비용을 줄인다

$\omega$  는 지연에 줄 가중치이고,  $1-\omega$  는 에너지에 줄 가중치다

| 수준         | 비용 표현                                  | 읽는 방법             |
|------------|----------------------------------------|-------------------|
| 단일 작업      | $\omega d + (1-\omega)e$               | 선택한 모드의 시간·에너지    |
| 시스템        | $\omega \sum D[t] + (1-\omega) \sum e$ | 슬롯 최대 지연 + 단말 에너지 |
| 큰 $\omega$ | 지연에 더 큰 비중                             | 에너지가 늘 수도 있음      |
| 모델 범위      | 작업 전체를 한 위치에서                          | 분할·반환 통신은 제외      |

$\omega = 0.5$ 는 선택한 수치 단위에서의 가중치이지 물리적으로 같은 비중의 보장이 아니다

# TOICA 한 슬롯: 서버 선택 뒤 채널 배분이 이어진다

DTO의 행동을 DCA로 평가한 결과가 다시 DTO 학습의 보상이 된다

|    |       |                   |
|----|-------|-------------------|
| 01 | 관측    | 단말 위치와 현재 작업의 크기  |
| ↓  |       |                   |
| 02 | 위치 선택 | DTO: 로컬 또는 한 서버   |
| ↓  |       |                   |
| 03 | 자원 배분 | DCA: 서버별 채널 수와 ID |
| ↓  |       |                   |
| 04 | 비용 평가 | 전송·계산 후 보상과 다음 상태 |

학습과 최적화가 결합된 방식이며, 모든 결정을 하나의 신경망이 출력하지 않는다

# DCA: 한 단말에 줄 채널 개수를 후보로 만든다

서버  $m$ 에 접속한  $N_m$ 개 단말과  $K$ 개 채널을 놓고 배분한다

| 기호        | 의미                  | 제약                |
|-----------|---------------------|-------------------|
| $z$       | 한 단말에 줄 채널 수        | 1부터 $K - N_m + 1$ |
| $v(n, z)$ | $z$ 개를 줄 때 추정 통신 비용 | 후보별 주어진 값         |
| $y(n, z)$ | 그 후보를 선택했는가         | 단말마다 정확히 하나       |
| 합계        | 선택된 $z$ 의 합         | 정확히 $K$           |

먼저 오프로딩 단말 집합을 고정한 뒤 그 집합 안에서 채널 수를 최적화한다

# 배분 예: 4개 채널을 두 단말에 모두 나눈다

설명용 비용표: 숫자는 작은 것이 좋고, 가능한  $z$ 는 1·2·3이다

| 채널 수 $z$ | 단말 A의 비용 | 단말 B의 비용 |
|----------|----------|----------|
| 1        | 24       | 6        |
| 2        | 12       | 3        |
| 3        | 8        | 2        |
|          |          |          |
|          |          |          |

모든 단말에게 같은 수를 주는 것이 최소 비용 배분은 아닐 수 있다

# 가능한 배분을 계산하면 A에 3개, B에 1개

K=4, Nm=2에서 모든 유효한 채널 개수 조합을 비교한다

| A / B 채널 수 | 합산 비용       | 판정       |
|------------|-------------|----------|
| 1 / 3      | $24+2 = 26$ | 가능       |
| 2 / 2      | $12+3 = 15$ | 가능       |
| 3 / 1      | $8+6 = 14$  | 이 표의 최솟값 |
| 4 / 0      | B에 채널이 없음   | 제약 위반    |

A에 한 채널을 더 주는 이득이 B에서 한 채널을 줄이는 손실보다 크다

# 동적 계획법: 앞 단말들의 최솟값을 재사용한다

$dp[n,j]$ : 앞  $n$ 개 단말에 정확히  $j$ 개 채널을 쓴 최소 비용

| 단계         | 계산 또는 상태                  | 값            |
|------------|---------------------------|--------------|
| 시작         | $dp[0,0]=0$ , 나머지 불가      | 불가는 $\infty$ |
| A만         | $dp[1,1..3]$              | 24 / 12 / 8  |
| A·B, 채널 4개 | $\min(24+2, 12+3, 8+6)$   | $dp[2,4]=14$ |
| 역추적        | B에 1개 $\rightarrow$ A에 3개 | 총 4개         |

DP는 채널을 임의로 조금씩 주는 것이 아니라 모든 유효한 후보의 최솟값을 누적한다

# 채널 3개와 채널 ID 1·2·4는 다른 정보다

DCA의 DP는 개수를 구하고, 구체적 채널은 그 개수에 맞춰 무작위로 붙인다

| 단계      | 예                      | 의미         |
|---------|------------------------|------------|
| 개수 최적화  | $A=3, B=1$             | 비용표 기준 결과  |
| ID 배정 예 | $A=\{1,2,4\}, B=\{3\}$ | 가능한 한 배정   |
| 같은 셀    | 서로 겹치는 ID 없음           | 셀 안 충돌 방지  |
| 다른 셀    | ID 1을 재사용 가능           | 셀 간 간섭은 남음 |

배분 개수의 최적성과 특정 주파수 조합의 최적성을 구분한다

# 채널보다 단말이 많으면 제약부터 충돌한다

각 단말 최소 1개와 총 K개 사용을 동시에 만족하려면  $N_m \leq K$ 여야 한다

| 상황           | 가능성         | 필요한 해석        |
|--------------|-------------|---------------|
| K=4, $N_m=2$ | 가능          | z 후보는 1..3    |
| K=4, $N_m=4$ | 각 단말 1개     | 배분 자유도 없음     |
| K=4, $N_m=5$ | 현재 제약에서는 불가 | 접속·오프로딩 결정 조정 |
| 오프로딩 단말 0개   | 활성 배분 대상 없음 | 빈 서버 처리 별도    |

서버를 선택하는 것과 자원 제약 아래 그 선택을 실행하는 것은 다르다

# Q값: 지금 행동의 장기 보상을 예측한다

DQN은 행동별 기대 보상을 표 대신 신경망으로 추정하는 방식이다

| 개념       | 뜻             | TOICA에서의 역할 |
|----------|---------------|-------------|
| 상태 $s$   | 현재 결정에 쓰는 정보  | 위치·현재 작업    |
| 행동 $a$   | 지금 선택할 실행 위치  | 로컬 또는 서버    |
| 보상 $r$   | 선택 결과의 개선 신호  | 로컬 대비 비용 감소 |
| $Q(s,a)$ | 미래 보상까지 고려한 값 | 행동 간 비교     |

Q값이 큰 행동을 고르는 것이 목표이며, Q값 자체가 초나 줄은 아니다

# DTO: 현재 작업의 상태로 로컬 또는 서버를 고른다

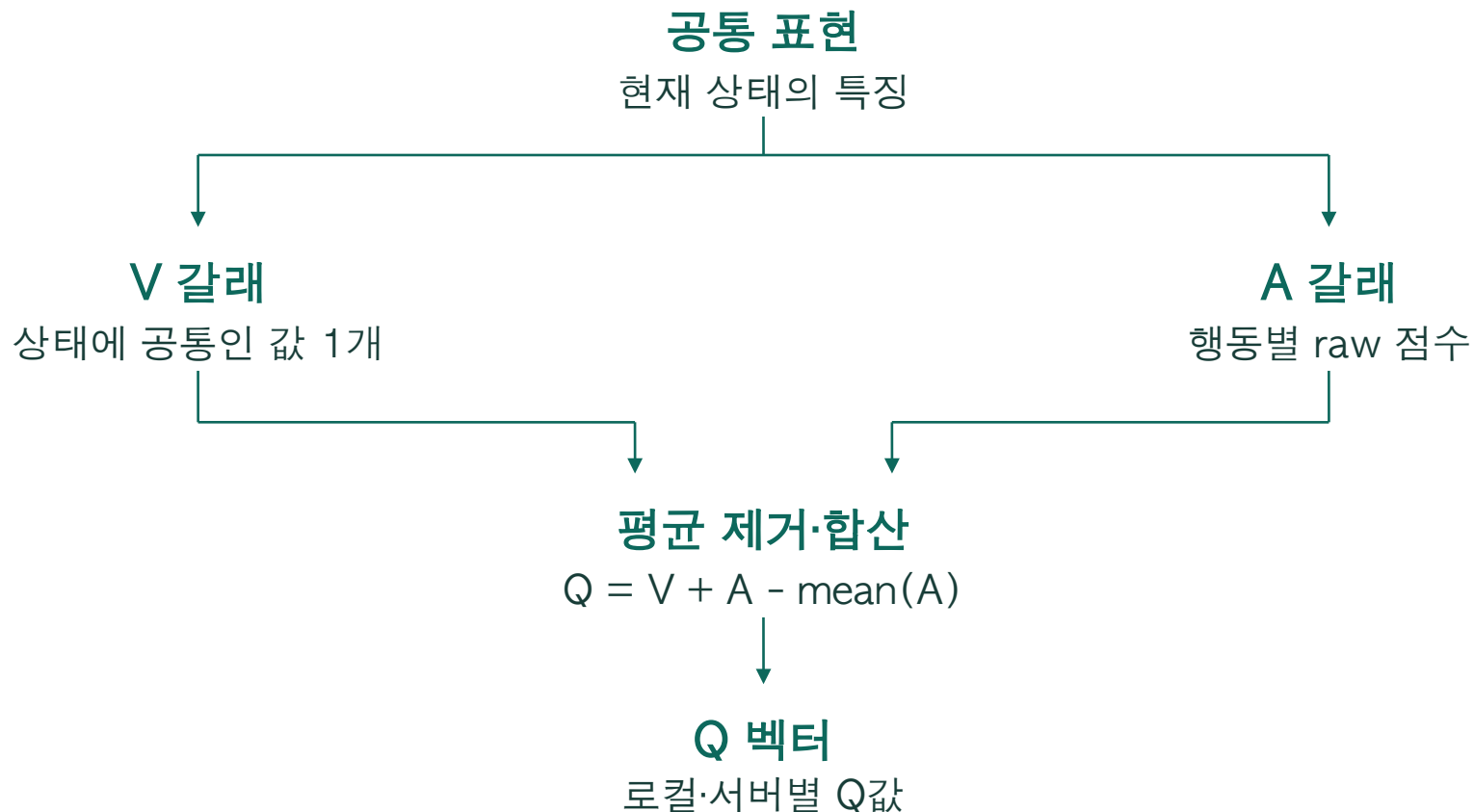
각 단말은  $s = \{\text{현재 위치, 입력 크기, 계산량}\}$ 을 관측한다

| 구분          | 논문에 명시된 값          | 출력 해석          |
|-------------|--------------------|----------------|
| 위치 $X_n(t)$ | 단말의 현재 공간 위치       | 시간에 따라 변화      |
| 작업 $b / c$  | 입력 비트 / CPU cycles | 현재 작업의 요구      |
| 행동 $a=0$    | 단말에서 실행            | 채널 배분 불필요      |
| 행동 $a=m$    | 서버 $m$ 에서 실행       | DCA의 접속 집합에 포함 |

D3QN 출력 개수는 단말당  $M+1$ 이며, 채널 ID 전체 조합을 직접 출력하지 않는다

# Dueling: 상태 공통값과 행동별 차이를 나눈다

D3QN의 Q망 안에서는 두 출력 갈래가 다시 한 Q벡터로 합쳐진다



# Double DQN: 고르는 망과 평가하는 망을 구분한다

설명용 다음 상태: Main Q=[2,3], Target Q=[4,1], 보상 0.2,  $\gamma = 0.9$

| 계산            | 선택·평가        | 목표 y                       |
|---------------|--------------|----------------------------|
| Main에서 선택     | 두 번째 행동: Q=3 | argmax 행동만 전달              |
| Target에서 평가   | 그 행동의 값은 1   | $0.2 + 0.9 \times 1 = 1.1$ |
| Target 자체 max | 첫 번째 행동 값 4  | DQN이라면 3.8                 |

D3QN은 Dueling Q망을 사용하면서 Double DQN 방식으로 목표를 만든다

# 보상: 로컬 실행보다 얼마나 비용을 줄였는가

$r = \tanh(\text{로컬 비용} - \text{선택한 실행 비용})$ , DCA 결과가 선택 비용에 들어간다

| 설명용 비용 차이  | 보상 $r$   | 해석       |
|------------|----------|----------|
| +0.4       | 약 +0.380 | 로컬보다 개선  |
| 0          | 0        | 로컬과 같음   |
| -0.3       | 약 -0.291 | 로컬보다 나쁨  |
| $\tanh$ 변환 | -1과 1 사이 | 큰 차이를 압축 |

이 보상은 학습을 위한 개선 신호이며, 전역 최적해의 증명은 아니다

# 학습: 배분까지 끝난 전이를 경험으로 저장한다

DCA를 제외하고 DTO만 학습하면 평가하는 환경 자체가 달라진다

|    |           |                             |
|----|-----------|-----------------------------|
| 01 | 행동·배분     | $\epsilon$ -greedy 선택 후 DCA |
| ↓  |           |                             |
| 02 | 경험 저장     | 상태·행동·보상·다음 상태              |
| ↓  |           |                             |
| 03 | 표본 학습     | replay에서 batch를 뽑아 갱신       |
| ↓  |           |                             |
| 04 | Target 복사 | 정해진 주기에 Main을 복사            |

과거 경험의 재사용과 느리게 바꾸는 Target이 학습을 안정화하려는 장치다

# 학습과 실제 의사결정에서 필요한 계산을 나눈다

논문은 오프라인 학습과 온라인 의사결정의 두 단계를 구분한다

| 활동          | 학습 단계     | 온라인 단계         |
|-------------|-----------|----------------|
| DTO Q값 계산   | 필요        | 필요             |
| DCA 배분      | 매 환경 상호작용 | 현재 접속 집합에 필요   |
| Replay·망 갱신 | 경험으로 수행   | 추론만 할 때는 제외    |
| 상태·비용 측정    | 학습 환경의 입력 | 실제 수집·제어 연동 필요 |

오프라인으로 정책을 학습해도 온라인에서 상태 수집과 DCA 실행은 사라지지 않는다

# 최적화의 범위: DCA와 전체 TOICA를 구분한다

고정된 비용표의 최적 배분은 전체 시스템의 전역 최적성보다 좁은 주장이다

| 구성     | 직접 다루는 것       | 자동으로 보장하지 않는 것 |
|--------|----------------|----------------|
| 우선순위   | 추정 비용의 후속 경로   | 최적 작업 순서       |
| DCA    | 고정 $v$ 의 채널 개수 | ID별 간섭의 전역 최적  |
| DTO 보상 | 로컬 대비 tanh 개선  | 전역 makespan 최소 |
| 공동 실행  | 선택·배분을 매번 연결   | 모든 조건의 성능 보장   |

이 논문의 기여는 결합 설계와 조건부 실험 성과로 읽는다

# 비교군: 무엇을 바꾸었는지 알아야 결과를 읽는다

같은 DCA를 쓰는 비교와 DCA를 제거한 비교는 다른 질문에 답한다

| 비교군      | 바꾼 요소        | 점검하는 효과         |
|----------|--------------|-----------------|
| DTO only | 채널을 무작위로 배분  | DCA 기여          |
| TOICA-RA | 무작위 작업 우선순위  | 의존 작업 순서 규칙     |
| SEG+DCA  | greedy 위치 선택 | DTO와 greedy의 차이 |
| ON+DCA   | 가장 가까운 서버 선택 | 거리만 쓰는 선택과 차이   |

방법 이름만 비교하지 않고 작업 순서·위치 결정·자원 배분의 변경점을 나눈다

# 평가 환경: 5개 단말과 3개 서버의 시뮬레이션

기본값과 규모 확대 실험을 구분한다

| 항목       | 기본 설정                    | 의미                      |
|----------|--------------------------|-------------------------|
| 통신       | 50MHz / 8개 채널            | 서버별 배분 자원               |
| 단말 / 서버  | 5개 / 3개                  | 이동 반경 500m              |
| 작업       | 평균 15개 노드                | 150-400kB, 30-80Mcycles |
| 계산 / 가중치 | 서버 5GHz / $\omega = 0.5$ | 기본 비교 조건                |

하드웨어와 통신 수치는 평가 모델의 입력이며 실물 장치에서의 성능 보증이 아니다

# 결과: 자원 배분을 결합한 쪽의 비용이 낮았다

Fig.5-7은 제시된 시뮬레이션 설정 안에서의 비교다

| 그림      | 바뀐 조건            | 관찰               |
|---------|------------------|------------------|
| Fig.5   | DTO only와 비교     | TOICA 보상 더 높게 안정 |
| Fig.6   | 대역폭 30-70MHz     | 대역폭 증가 시 비용 감소   |
| Fig.7   | 서버 계산력 3-7GHz    | 계산력 증가 시 비용 감소   |
| 채널 수 비교 | 같은 대역폭의 K=8 / 16 | K=16의 비용이 더 낮음   |

그래프의 Cost Valuation은 원시 에너지 J나 지연 s가 아닌 변환된 비용이다

# 가중치와 규모를 바꾸면 성능과 학습도 바뀐다

규모 확대 그래프는 하나의 망을 아무 수정 없이 적용한 결과가 아니다

| 변경               | 관찰               | 해석 경계          |
|------------------|------------------|----------------|
| $\omega$ 증가      | 대체로 지연 ↓ · 에너지 ↑ | 개별 점은 엄격 단조 아님 |
| 작업 수 9 / 15 / 21 | 많을수록 비용 증가       | 같은 입력 난도 아님    |
| 서버 수 3 / 6 / 9   | 재학습 보상 곡선 제시     | 출력 크기 · 학습률 변경 |
| 단말 수 증가          | 자원 경쟁 · 전체 비용 증가 | 무제한 확장성 아님     |

확장 실험은 적응 가능성의 근거지만 동일 모델의 무수정 배포 보장과는 다르다

# 36.8ms는 무엇을 나누어 얻은 시간인가

논문은 1,000 episode 시뮬레이션에 551.5초를 보고한다

| 계산·범위       | 값                             | 판정                |
|-------------|-------------------------------|-------------------|
| 총 시뮬레이션     | 551.5s / 1,000 episode        | 약 0.5515s/episode |
| 15개 작업으로 나눔 | $0.5515 / 15 \approx 0.0368s$ | 약 36.8ms/작업       |
| 포함 범위       | 시뮬레이션 평균                      | 순수 추론 지연 아님       |
| 실시간 적용      | 실장치 수집·DCA·제어                 | 별도 end-to-end 검증  |

평균 시뮬레이션 시간을 실제 장치의 추론·마감시간 보장으로 옮기지 않는다

# 수식의 빈틈은 성과와 구분해서 읽는다

논문의 설명을 그대로 인용하는 것과 증명을 확인하는 것은 다르다

| 항목         | 인쇄된 내용            | 안전한 해석           |
|------------|-------------------|------------------|
| Eq. 12     | 평균이라 부르지만 합 표기    | 추정 비용 $q$ 로 설명   |
| NP-hard 주장 | 문제에서 GKP로 매핑      | 이 방향만으로 증명 안 됨   |
| DCA 연산 수   | $Nm(K-Nm+1)(K+1)$ | 경계 $Nm=K$ 에서도 양수 |
| 슬롯·무배정     | 합산식·빈 서버 예외 생략    | 구현 시 조건 명시 필요    |

비판은 제시된 증명·표기의 범위에 한정하며 전체 실험이 무효라고 단정하지 않는다

# 핵심 정리: 실행 위치와 자원 배분을 함께 본다

TOICA는 작업 순서, 서버 선택, 채널 배분을 연결하는 설계다

- |    |       |                     |
|----|-------|---------------------|
| 01 | 순서    | DAG 후속 비용으로 우선순위    |
| ↓  |       |                     |
| 02 | 위치    | D3QN으로 로컬·서버 선택     |
| ↓  |       |                     |
| 03 | 채널    | DCA로 수량 최적화 후 ID 배정 |
| ↓  |       |                     |
| 04 | 검증 범위 | 조건부 시뮬레이션과 한계       |

DCA의 부분 문제 최적성과 TOICA 전체의 조건부 성과를 함께 구분해 이해한다